



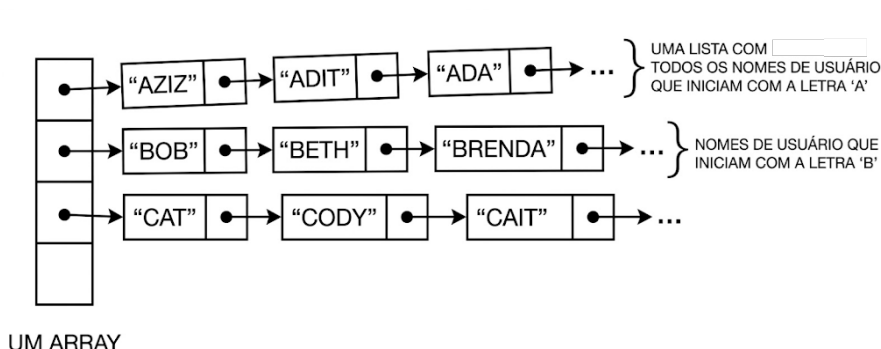
## Observações Gerais

- É vedado o uso de ferramentas de inteligência artificial ou modelos de linguagem (LLMs).
- A constatação de utilização de tais recursos implicará na atribuição de nota **zero**.
- O discente poderá ser chamado para apresentar e justificar o código desenvolvido.
- A reprodução parcial ou integral do trabalho de outro colega resultará igualmente em nota **zero**.
- Durante a aula, algumas resoluções dos alunos serão selecionadas por sorteio para apresentação e debate.

## Array de Listas

Na atividade anterior, criamos um sistema de agenda que armazenava os contatos em uma única lista (array). À medida que o número de contatos cresce, buscar, inserir ou remover uma pessoa em uma lista única pode se tornar ineficiente, pois é necessário percorrer muitos elementos sequencialmente. Para resolver esse problema, vamos evoluir o sistema utilizando uma estrutura híbrida: um vetor de listas (arrays).

Imaginemos a seguinte situação: para organizar a base de dados de uma plataforma, decidimos utilizar uma estrutura de dados híbrida formada por uma lista (array) principal composta por 26 posições (slots), onde cada posição corresponde a uma letra do alfabeto (de A a Z).



Cada uma dessas 26 posições aponta para outra lista (array) que contém os nomes dos usuários iniciados por aquela determinada letra:

- Posição 1 (Letra A): contém uma lista (array) com nomes como "AZIZ", "ADIT", "ADA", etc.
- Posição 2 (Letra B): contém uma lista (array) com nomes como "BOB", "BETH", "BRENDA", etc.
- Posição 3 (Letra C): contém uma lista (array) com nomes como "CAT", "CODY", "CAIT", etc.

Para otimizar a aplicação, você deverá evoluir o sistema utilizando uma estrutura híbrida: um array de tamanho 26 onde cada posição contém uma instância da classe **Vetor**.

## Implementação

### 1. Uso da Classe Vetor

- O array principal terá 26 posições (uma para cada letra do alfabeto).
- Em cada uma das 26 posições, você deve instanciar o Vetor (classe desenvolvida durante as aulas) para representar a lista de contatos daquela letra específica.

### 2. Mapeamento de Índice:

- Crie uma função auxiliar que receba o nome do contato (ou a primeira letra) e retorne o índice correto no array principal (ex: 'A' ou 'a' → índice 0, 'B' ou 'b' → índice 1, ..., 'Z' ou 'z' → índice 25).
- Sensibilidade a Maiúsculas e Minúsculas (Case Sensitivity): Garantir que "ana" e "Ana" sejam direcionadas para o mesmo slot no array (índice 0).

### 3. Menu Interativo: O programa deve rodar em um laço e oferecer as seguintes opções ao usuário:

- Adicionar Contato (Ordenado):** Determine o índice do array pela primeira letra do nome do contato e insira-o de forma ordenada por nome dentro do Vetor daquela posição.
- Remover Contato:** Localize o índice correspondente à primeira letra do nome, busque o contato dentro do Vetor e remova-o.
- Buscar Contato:** Acesse diretamente a posição do array referente à primeira letra e realize a busca apenas dentro daquele Vetor.
- Atualizar Contato:** Busque o contato na lista correspondente e atualize suas informações.
- Sair:** Finaliza a execução da aplicação

```
--- MENU DA AGENDA DE CONTATOS ---
1 - Adicionar Contato (Ordenado)
2 - Remover Contato
3 - Buscar Contato
4 - Atualizar Contato
5 - Sair

Escolha uma opcao: 1

Digite o nome do contato: Ana Silva
Digite o telefone: (11) 98765-4321
Contato 'Ana Silva' adicionado com sucesso na letra 'A' (Indice 0)!

Escolha uma opcao: 1
Digite o nome do contato: Adit B
Digite o telefone: (11) 91234-5678
Contato 'Adit B' inserido ordenadamente na letra 'A' (Indice 0)!

Escolha uma opcao: 3
Digite o nome do contato a buscar: Ana Silva
Contato encontrado no Vetor 'A': Ana Silva, Telefone: (11) 98765-4321

Escolha uma opcao: 4
Digite o nome do contato a atualizar: Ana Silva
Digite o novo telefone: (11) 99999-8888
Contato 'Ana Silva' atualizado com sucesso!

Escolha uma opcao: 2
Digite o nome do contato a remover: Adit B
Contato 'Adit B' removido com sucesso do Vetor 'A'!
```

## Questões para Responder na Entrega:

1. Por que essa abordagem usando um array de instâncias da classe Vetor torna a busca de um contato mais rápida em comparação com a versão anterior (uma única lista contendo todos os contatos)?
2. O que acontece com o desempenho da busca se a maioria dos contatos cadastrados começar com a mesma letra (ex: centenas de nomes iniciando com a letra "M")? O sistema continuará rápido? Justifique.